Input-Adaptive Quantization for Latency-Constrained Domain

1st Aditya Mahajan
CSC department
NCSU
Raleigh, NCSU
amahaja8@ncsu.edu

***Abstract*—Dynamic quantization methods offer accuracy advantages over static quantization by adapting bit-width assignments to each inference input. However, existing state-of-the-art methods report pseudo-latency metrics such as BitOPs that do not reliably reflect real-world inference latency. In this work, we implement DQNet, a dynamic quantization method using a parallel bit-controller network, and successfully replicate its results on CIFAR-10 with ResNet-20. We demonstrate that despite favorable BitOPs, DQNet incurs substantial wall-clock latency overhead compared to static quantization baselines. Motivated by this gap, we propose and evaluate several modifications aimed at reducing real latency while preserving the accuracy benefits of dynamic quantization. Our primary contribution, a global routing scheme that directs entire images to a single uniform bit-width, achieves nearly half the latency of DQNet while maintaining competitive accuracy and a smaller model size than static INT8 quantization. We further investigate early exit and block skipping strategies, finding that these techniques do not yield meaningful gains at the scale of ResNet-20 on CIFAR-10, though they remain promising for larger architectures.**

***Index Terms*—quantization, dynamic quantization, neural network compression, inference latency, ResNet, CIFAR-10**

I. Introduction

Dynamic or Input specific quantization is now a commonly used technique as a model compression technique [10]. Unlike static quantization, the specification of the quantization for the model are run at inference time, aiming to exploit patterns within the inference inputs to squeeze as much compression while maintaining accuracy. This differs with static quantization methods, where there is no change dependent on inputs. We have implemented DQNet [1], a dynamic quantization method which uses a parallel bit controller net. The purpose of this net is to set the quantization bits of the main model. We are successful in replicating the results of the DQNet paper on the CIFAR-10 dataset [3], using the base model of ResNet-20 [2]. We also contrast its performance vs simple static quantization methods, to more accurately understand the payoffs. However, dynamic quantization methods have the obvious drawback of greater overhead. Not only are their implementation more complicated and computationally expensive, the additional pre-inference calculation also have to be considered. Existing SOTA papers [1], [9] generally only report FLOPs, BitOPs or similar pseudo-latency metrics. We will see that these do not necessarily translate to real latency gains. Hence we can make a case for existing SOTA dynamic quantization methods not being ideal for scenarios where speed of inference is the bottleneck. We propose improvements over DQNet, with the aim of utilizing both the accuracy edge of dynamic quantization, while reducing latency as much as possible. We have detailed several experiments, and contrasted them with our DQNet implementation.

II. DQNet Implementation

A. Baselines chosen

1) Model: The baseline is a ResNet-20 architecture [2], a well-studied convolutional network with 20 weight layers organized into residual blocks. Each block contains two 3x3 convolutions with batch normalization [11] and ReLU activations, connected by an identity shortcut that enables gradient flow and residual learning. The network uses three groups of blocks operating at spatial resolutions 32x32, 16x16, and 8x8 with channel widths of 16, 32, and 64 respectively. This model was chosen due being well studied, while being relatively modest in size, allowing for many experimental runs.

2) Dataset: The dataset used was CIFAR-10 [3], due to its widespread usage across the literature, and its modest size.

B. DQNet explanation

1) Overview: DQNet [1] assigns a per-layer bit-width to each input image independently. A lightweight bit-controller network examines the raw input image and produces probability distributions over the candidate bit-widths (e.g., 3, 4, 5, 6, 7 bits) for each of the 10 controlled layers (1 initial conv + 9 residual blocks). The controller and backbone are trained jointly end-to-end.

2) Bit-Width Selection: The fundamental challenge is that bit-width selection is a discrete decision, which breaks gradient-based optimization. This is addressed through two mechanisms:

- Gumbel-Softmax Relaxation [4]: During training, the controller's logits are passed through the Gumbel-Softmax function, which produces a continuous approximation to categorical sampling. A temperature parameter controls the sharpness of the distribution: high temperature produces nearly uniform weights (exploration), while low temperature approaches a one-hot vector (exploitation). Temperature is linearly annealed from 1.0 to 0.1 over the course of training.

- Straight-Through Estimator (STE) [5]: The quantization function itself (rounding) is non-differentiable. STE resolves this by using the quantized values in the forward pass but passing gradients through as if quantization were the identity function.

3) Training: The computational cost of a quantized convolution scales quadratically with the bit-width (since both weights and activations are quantized). The effective cost in Bit Operations ($BitOPs$) for a layer with FLOPs F at bit-width b is:

$$BitOPs = b^2 \cdot F$$

The total expected $BitOPs$ across all layers is:

$$Total = \sum_{layers} \left(\sum_{bits} p(b) \cdot b^2 \cdot F_{layer} \right)$$

where $p(b)$ is the probability assigned to bit-width b by the controller.

The model is trained with a composite loss:

$$L = L_{CE} + \lambda \cdot \left(\frac{BitOPs}{BitOPs_{max}} \right)$$

where L_{CE} is the standard cross-entropy classification loss, $BitOPs_{max}$ is the cost when all layers use the maximum bit-width, and λ is a hyperparameter (set to 0.0001) controlling the accuracy-efficiency trade-off. This encourages the controller to use lower bit-widths where possible without significantly harming accuracy.

C. Our implementation

1) Architecture for Bit controller: The controller is deliberately lightweight to minimize its own overhead. It consists of two 3x3 convolutional layers (3 -¿ 16 -¿ 32 channels) with stride 2 and ReLU, followed by adaptive average pooling and a set of independent linear heads (one per controlled layer). This produces per-layer logits from the raw input image with minimal computation.

Initially, the architecture was even smaller, but it was found that too small a controller leads to 'static collapse'. This is when the model defaults to a configuration of quantization bits regardless of the inference input, leading it to be no better than static quantization, but with additional overhead cost.

TABLE I
COMPARISON OF MODELS

Model	BitFLOPs	Acc (%)
Baseline	8.51G	91.74
Our DQNet	0.97G	91.68
Paper DQNet	0.98G	92.01

2) Comparison of results: As we can see from the above table, our implementation is quite close to the original paper's implementation [1]. The slight discrepancy in accuracy is explained due to our simpler bit controller net.

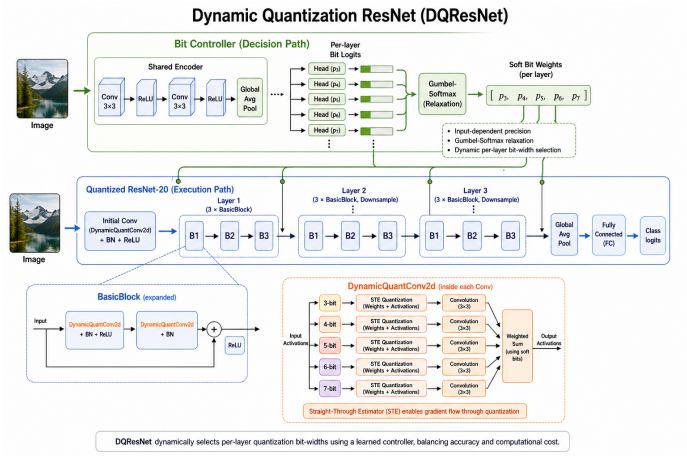

Fig. 1. Diagram of the Overall Architecture

III. EXPERIMENTS

A. Motivation

While DQNet offers fine-grained per-layer control, it incurs significant latency overhead: every layer must run convolutions at all candidate bit-widths during the forward pass (even though only one is "selected").

We aim to modify our existing architecture to minimize latency while maintaining accuracy. We will use static quantization as a baseline, since it has minimal overhead, but is usually worse than dynamic quantization. Our goal is to find a "middle ground" which preserves the gain in accuracy from dynamic quantization, as well as the speed of static quantization.

B. Evaluation

All methods are evaluated on the following axes:

- Test Accuracy (%): Classification accuracy on the CIFAR-10 test set (10,000 images).
- Inference Latency (seconds): Wall-clock time to run the full test set, averaged over 30 runs with a warmup pass. GPU synchronization is used where applicable.
- Model Size (MB): Theoretical memory footprint at the effective average bit-width.

C. Experiment 1: Global Routing

1) Approach: A logistic regression router (single linear layer on the flattened 3x32x32 input) classifies each image as "Easy" (INT5) or "Hard" (INT7). This is conceptually related to mixture-of-experts routing [12], where a gating function directs inputs to specialized sub-networks. The same Gumbel-Softmax training mechanism [4] is used, but now there is only one routing decision per image rather than one per layer.

2) Inference Efficiency: The key latency advantage emerges at inference time. Because the decision is global, the batch can be physically split into two sub-batches:

- Easy images run through the entire network at INT5.
- Hard images run through the entire network at INT7.

Each sub-batch executes only ONE quantized convolution per layer (not multiple), eliminating the overhead of running all bit-width options. This produces a direct latency reduction proportional to the split.

Additionally, the descision mechanism can be simplified due to the choices being far simpler, i.e., a single choice rather than 1 choice per layer. This enables us to reduce the overhead of the controller net.

3) Trade offs: The trade-off is reduced flexibility: all layers in a given image use the same bit-width. However, for many practical deployments, the latency savings from single-convolution execution outweigh the marginal accuracy benefit of per-layer control.

4) Results: This experiment was a success. We were able to maintain accuracy and model size close to DQNet, with a substantial latency improvement.

TABLE II
COMPARISON OF ACCURACY, LATENCY, FLOPS, AND MODEL SIZE

Model	Accuracy (%)	Latency (s)	Size (MB)
Baseline FP32	91.74	0.2794	1.04
Static INT8	91.48	0.5637	0.26
DQNet	91.68	1.3290	0.17
Global Routing	91.28	0.6869	0.20

As we can see from the results, latency was nearly halved from DQNet. We can also note that as expected, accuracy and latency are slightly worse than the static quantization baseline, However this is offset by the significantly smaller model size.

D. Experiment 2: Early Exit

1) Approach: Instead of varying the precision of computation, this experiment reduces the AMOUNT of computation by allowing confident predictions to exit the network early, dependent on the inference input. This approach draws on prior work in early exit networks [6] and adaptive computation time [7], [8]. This is complementary to quantization, and therefore, can be implemented in parallel.

2) Architecture: An auxiliary classifier head is attached after the second residual group (layer2). This head consists of adaptive average pooling followed by a linear layer. During training, BOTH the early and final classifiers are trained simultaneously with a weighted loss:

$L = L_final + alpha * L_early$

where alpha = 0.3 balances the two objectives. This ensures the early classifier produces useful predictions without degrading the final classifier's performance.

At inference, the early classifier's output is converted to a probability distribution via softmax, and its entropy is computed:

$H = -sum(p_i * log(p_i))$

Low entropy indicates a peaked distribution (high confidence). If H is below a user-specified threshold, the sample exits early using the auxiliary classifier's prediction. Otherwise, it continues through layer3 and uses the final classifier.

3) Threshold sweep: The entropy threshold is a tunable operating point. A sweep across thresholds produces a Pareto frontier of accuracy vs. latency:

- Low threshold: Few early exits, accuracy close to baseline, minimal latency reduction.
- High threshold: Many early exits, potentially degraded accuracy, significant latency savings.

4) Results: This experiment was unfortunately not as successful. For a model of this size, early exiting causes massive accuracy losses. Also, there are not many choices of when to exit. There is no opportunity to fine tune the location of the exit head. We can conclude that while this method may be useful in larger models, we can not show meaningful gains for our smaller experimental setup.

E. Experiment 3: Block skipping

1) Approach: This approach is related to prior work on conditional computation in residual networks, such as SkipNet [13] and spatially adaptive computation [8]. Unlike those methods, our approach requires no learned gating parameters.

After the first convolution in a residual block (conv1 , bn1 , relu), some fraction of activations are zero (killed by ReLU). If a large fraction of activations are zero, the intermediate representation carries little information, and the second convolution (conv2) is unlikely to contribute meaningfully to the output. In a residual block [2] with an identity shortcut, the block computes:

$y = relu(conv2(relu(conv1(x))) + x)$

If conv2 contributes little, the output is approximately relu(x) x (since x has already passed through previous ReLU layers).

2) Mechanism: The "activity level" is defined as the fraction of non-zero activations after relu(bn1(conv1(x))):

$activity = mean(1[out1 > 0])$

where the mean is taken over channels and spatial dimensions. If activity is below a threshold tau, the block ABORTS: conv2 is skipped, and the output is simply the identity shortcut (x). This applies only to non-downsample blocks (where the shortcut is the identity, not a 1x1 conv)

3) Implementation: Because no new parameters are introduced, the pre-trained baseline model can be loaded directly into the activity-gated architecture and evaluated at any threshold without retraining. This produces a sweep of operating points trading accuracy for latency savings.

4) Results: Similar to experiment 2, the experiment did not yield the desired effects. ResNet-20 is too short too have many meaningful "zeroed out" blocks, meaning we do not achieve any benefits unless our threshold is low, in which case we suffer tremendous accuracy losses. Again, we can conclude that while this method is useful for longer neural networks [13], [14], we cannot show meaningful gains from our experimental setup.

IV. CONCLUSION

This work examined the real-world latency implications of dynamic quantization through a reimplementation of DQNet

[1] on ResNet-20 [2] and CIFAR-10 [3]. Our reproduction closely matches prior results, achieving 91.68% accuracy at 0.97G BitFLOPs, validating the correctness of our implementation and highlighting the importance of sufficient controller capacity to avoid collapse into static behavior.

A central finding is that BitFLOPs are a poor proxy for actual inference latency. Despite a substantial reduction in theoretical compute, DQNet incurs significantly higher wall-clock latency than both FP32 and static INT8 baselines due to its multi-path execution overhead. This suggests that efficiency claims based solely on compute metrics can be misleading in practical deployments.

To address this, we proposed a global routing scheme that assigns a single bit-width per input. This removes multi-branch execution and enables true single-pass inference. The result is a substantial latency reduction—nearly half that of DQNet—while maintaining competitive accuracy and a smaller model size than static INT8. This demonstrates that coarse-grained adaptivity can capture much of the benefit of dynamic quantization without its overhead.

In contrast, early exit [6] and block skipping [13] did not yield meaningful gains in this setting. The limited depth of ResNet-20 restricts their effectiveness, indicating that such methods are better suited to larger architectures.

Overall, this work underscores the importance of evaluating quantization methods using real latency metrics. It also shows that simpler, hardware-aligned strategies like global routing can offer a more practical balance between accuracy, efficiency, and deployability than fine-grained dynamic approaches.

V. GITHUB REPOSITORY

All code can be found at https://github.com/MrRoboto11102003/Quantization_project.git

VI. GRAPHS

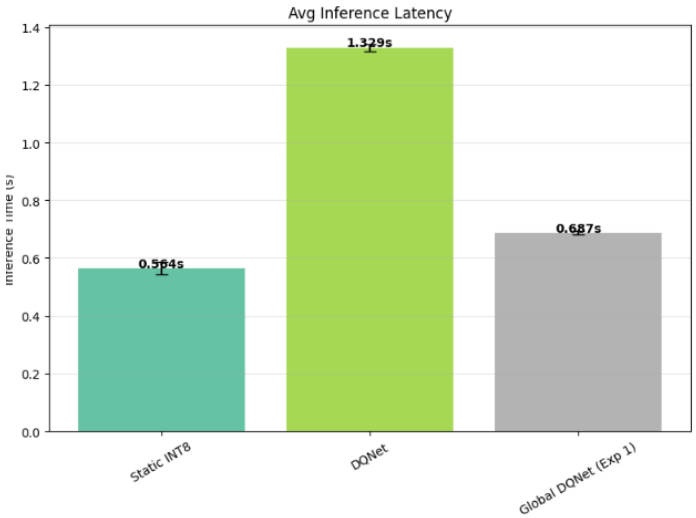

Fig. 2. Comparison of Latencies, with a static baseline, DQNet, and experiment 1

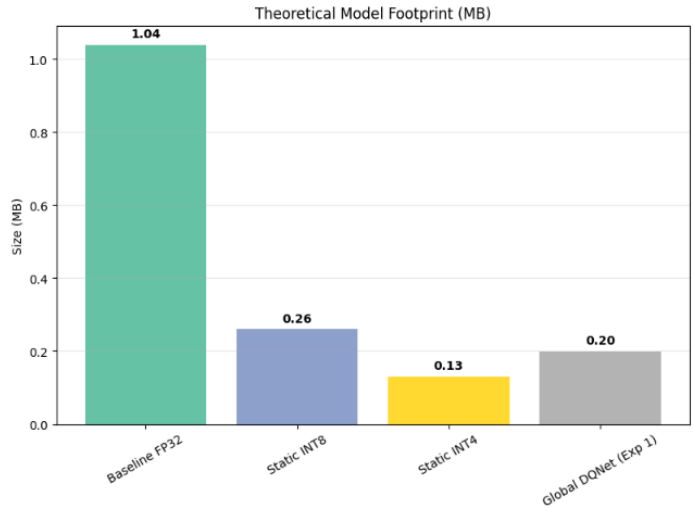

Fig. 3. Comparison of model size, with a static baselines, and experiment 1

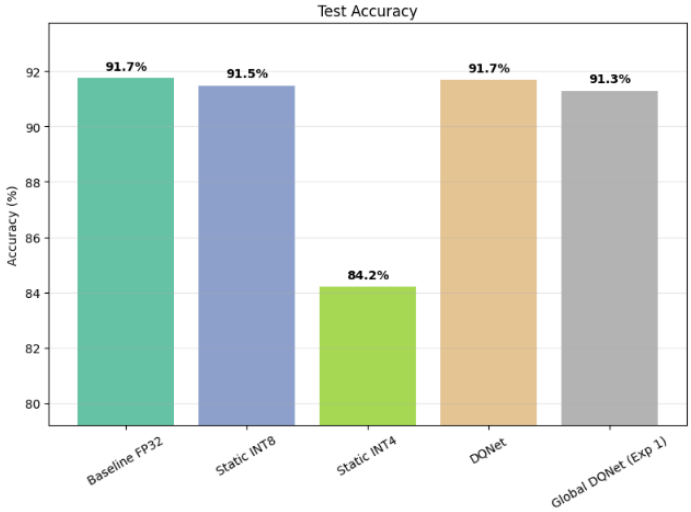

Fig. 4. Comparison of model accuracy, with a static baselines, DQNet and experiment 1

REFERENCES

[1] Z. Liu, Y. Wang, K. Han, S. Ma, and W. Gao, "Instance-Aware Dynamic Neural Network Quantization," in *Proc. IEEE/CVF Conf. on Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 12279–12288.

[2] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in *Proc. IEEE Conf. on Computer Vision and Pattern Recognition (CVPR)*, Las Vegas, NV, Jun. 2016, pp. 770–778.

[3] A. Krizhevsky, "Learning Multiple Layers of Features from Tiny Images," Technical Report, University of Toronto, 2009.

[4] E. Jang, S. Gu, and B. Poole, "Categorical Reparameterization with Gumbel-Softmax," in *Proc. Int. Conf. on Learning Representations (ICLR)*, 2017.

[5] Y. Bengio, N. Léonard, and A. Courville, "Estimating or Propagating Gradients Through Stochastic Neurons for Conditional Computation," *arXiv preprint arXiv:1308.3432*, 2013.

[6] S. Teerapittayanon, B. McDanel, and H. T. Kung, "BranchyNet: Fast Inference via Early Exiting from Deep Neural Networks," in *Proc. 23rd Int. Conf. on Pattern Recognition (ICPR)*, Cancun, Mexico, Dec. 2016, pp. 2464–2469.

[7] A. Graves, "Adaptive Computation Time for Recurrent Neural Networks," *arXiv preprint arXiv:1603.08983*, 2016.

[8] M. Figurnov, M. D. Collins, Y. Zhu, L. Zhang, J. Huang, D. Vetrov, and R. Salakhutdinov, "Spatially Adaptive Computation Time for Residual Networks," in *Proc. IEEE Conf. on Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 6489–6498.

[9] Y. Xu, X. Dong, E. C. Tseng, J. Keuper, and R. Yu, "DNQ: Dynamic Network Quantization," in *Proc. Data Compression Conf. (DCC)*, 2019, pp. 429–429.

[10] A. Gholami, S. Kim, Z. Dong, Z. Yao, M. W. Mahoney, and K. Keutzer, "A Survey of Quantization Methods for Efficient Neural Network Inference," in *Int. J. Computer Vision*, vol. 130, no. 3, pp. 535–560, 2022.

[11] S. Ioffe and C. Szegedy, "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift," in *Proc. Int. Conf. on Machine Learning (ICML)*, 2015, pp. 448–456.

[12] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean, "Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer," in *Proc. Int. Conf. on Learning Representations (ICLR)*, 2017.

[13] X. Wang, F. Yu, Z.-Y. Dou, T. Darrell, and J. E. Gonzalez, "SkipNet: Learning Dynamic Routing in Convolutional Networks," in *Proc. European Conf. on Computer Vision (ECCV)*, 2018, pp. 409–424.

[14] Y. Han, G. Huang, S. Song, L. Yang, H. Wang, and Y. Wang, "Dynamic Neural Networks: A Survey," *IEEE Trans. Pattern Analysis and Machine Intelligence*, vol. 44, no. 11, pp. 7436–7456, 2022.